Avisa Aí: especificação do back-end

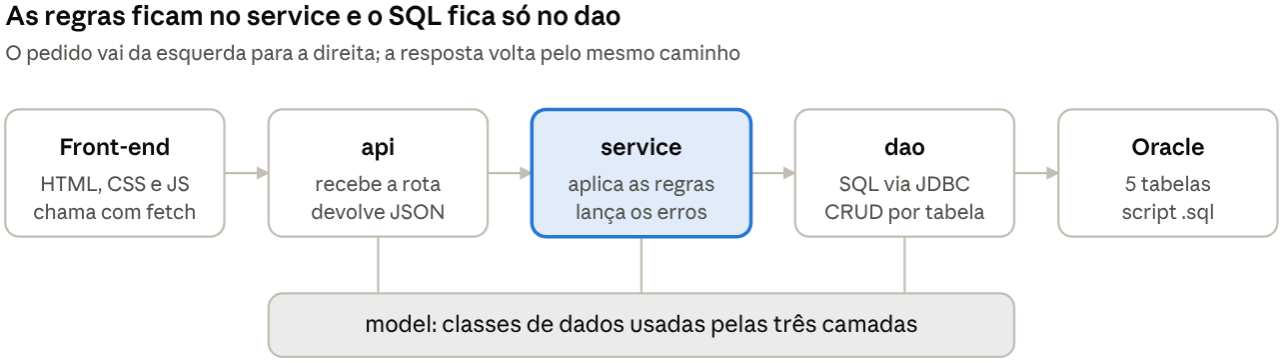
Nov 11, 2026 · @Letícia

O que entregar

Item da rubrica	Pontos	O que precisa existir
CRUD completo via JDBC	3	Inserir, listar, buscar, atualizar e excluir para as entidades principais
POO de verdade	1	3 ou mais entidades relacionadas, encapsuladas, com herança
Camadas separadas	1	Pacotes `model`, `dao`, `service` e `api`
Tratamento de erros	1	Exceções de banco e entradas inválidas com mensagem clara, sem o sistema cair
Diagrama de classes (UML)	1	Feito pela dupla do pitch a partir do seu código; avise quando as classes estiverem estáveis

Regras do enunciado que afetam o back: só vale tecnologia vista até a fase 7, o acesso ao banco é por JDBC, e código que não compila ou não conecta no banco zera o requisito.

Arquitetura



arquitetura · 4 pacotes Java entre o front e o Oracle

Cada camada só chama a vizinha da direita: o handler da `api` nunca acessa o DAO direto, e o DAO nunca decide regra.

Estrutura de pastas, dentro de `backend/src/br/com/avisaai/`:

Pacote	Conteúdo
(raiz)	`Main.java`, que sobe o servidor
`model`	Classes de dados e o enum de status
`dao`	`ConnectionFactory` e um DAO por tabela
`service`	Um service por entidade, com as regras, e as exceções próprias
`api`	Um handler por rota e a classe utilitária de JSON

O enunciado chama a última camada de "interface", mas `interface` é palavra reservada do Java e não pode ser nome de pacote; por isso `api`.

Classes do model

O pacote `model` tem sete classes e um enum. Todos os atributos são `private`, com getters e setters.

Classe	Atributos	Observações
`Usuario` (abstrata)	`id`, `nome`, `email`, `bairro` (`Bairro`)	Métodos abstratos `getTipo()`, `podeApoiar()`, `podeMudarStatus()` e `podeEditar(Ocorrencia)`
`Cidadao` (herda de `Usuario`)	Os herdados	Tipo CIDADAO; apoia; não muda status; edita só as ocorrências de que é autor
`Agente` (herda de `Usuario`)	Os herdados	Tipo AGENTE; não apoia; muda status; edita todas
`Bairro`	`id`, `nome`	Sem regras próprias
`Categoria`	`id`, `nome`	Sem regras próprias
`Ocorrencia`	`id`, `titulo`, `descricao`, `endereco`, `status`, `dataAbertura`, `autor` (`Usuario`), `categoria`, `bairro`, `totalApoios`	`totalApoios` não é coluna: vem de um `COUNT` na consulta

Classe	Atributos	Observações
`Apoio`	`usuario`, `ocorrencia`, `dataApoio`	Representa a tabela associativa
`StatusOcorrencia` (enum)	ABERTA, EM_ANDAMENTO, RESOLVIDA	Mesmos valores do `CHECK` no banco

Pontos de atenção:

- **Herança com comportamento:** as permissões ficam nos métodos de `Cidadao` e `Agente`. Assim o service pergunta `usuario.podeMudarStatus()` em vez de comparar textos, e a herança tem motivo para existir.
- **Relacionamentos como objetos:** `Ocorrencia` guarda um `Usuario`, uma `Categoria` e um `Bairro`, e não só os ids.
- **Leitura do banco:** ao montar um usuário a partir do `ResultSet`, a coluna `TIPO` decide se o objeto criado é `Cidadao` ou `Agente`.
- **JSON:** cada classe tem um método `toJson()` que devolve o texto no formato descrito na seção de rotas.
- **Nomes das colunas:** os do documento de especificação do banco (`ID_USUARIO`, `ID_BAIRRO`, `DATA_ABERTURA` etc.).

Conexão e DAOs

O pacote `dao` é o único lugar do projeto com SQL e com classes do `java.sql`.

Conexão

- **`ConnectionFactory`:** lê `db.url`, `db.user` e `db.password` do arquivo `backend/db.properties` e devolve uma `Connection`.
- **Senha fora do código:** o `db.properties` real está no `.gitignore`; no repositório fica só o `db.properties.example`.
- **Driver:** o `ojdbc` fica em `backend/lib/` e o README explica como incluí-lo.
- **Arquivo ausente:** se o `db.properties` não existir, o sistema mostra uma mensagem dizendo como criá-lo, em vez de um erro técnico.

Métodos de cada DAO

DAO	Métodos
`BairroDao`	`inserir`, `listar`, `buscarPorId`, `atualizar`, `excluir`
`CategoriaDao`	`inserir`, `listar`, `buscarPorId`, `atualizar`, `excluir`
`UsuarioDao`	`inserir`, `listar`, `buscarPorId`, `atualizar`, `excluir`
`OcorrenciaDao`	`inserir`, `listar(idBairro, status)`, `buscarPorId`, `atualizar`, `atualizarStatus`, `excluir`
`ApoioDao`	`inserir`, `excluir`, `existe(idUsuario, idOcorrencia)`
`RelatorioDao`	`porBairro`, `porCategoria`, `rankingApoios`, `porStatus`

Regras de implementação

1. **Sempre `PreparedStatement`** com `?`, nunca SQL montado com concatenação de texto.
2. **`try-with-resources`** para fechar `Connection`, `PreparedStatement` e `ResultSet`.
3. **Listagem de ocorrências com `JOIN`:** a consulta já traz nome do autor, da categoria e do bairro e o total de apoios, para o front não precisar de várias chamadas.
4. **Filtros opcionais:** `listar(idBairro, status)` aceita `null` em qualquer um dos dois.
5. **Id gerado:** depois do `INSERT`, recuperar o id criado pelo banco e colocá-lo no objeto.
6. **Erros:** o DAO não imprime nem engole `SQLException`; ele a repassa embrulhada em `DadosException` (seção de erros).
7. **Relatórios:** as quatro consultas vêm prontas do responsável pelo SQL, no arquivo `sql/relatorios.sql`.

Services

O pacote `service` valida os dados e aplica as regras antes de chamar o DAO. Quando uma regra é violada, o service lança `RegraNegocioException` com a mensagem que vai aparecer na tela.

Regra	Service	Mensagem quando violada
Título, endereço, categoria e bairro são obrigatórios	`OcorrenciaService`	"Preencha título, endereço, categoria e bairro."
Toda ocorrência nasce ABERTA, com a data do dia	`OcorrenciaService`	Não se aplica: o service define os valores
Só agente muda o status	`OcorrenciaService`	"Apenas agentes podem alterar o status."
O status precisa ser um dos três valores do enum	`OcorrenciaService`	"Status inválido."
Cidadão edita e exclui só as próprias ocorrências	`OcorrenciaService`	"Você só pode alterar ocorrências que registrou."
Só cidadão apoia	`ApoioService`	"Agentes não apoiam ocorrências."
Ninguém apoia a própria ocorrência	`ApoioService`	"Você não pode apoiar a própria ocorrência."
Um apoio por cidadão em cada ocorrência	`ApoioService`	"Você já apoiou esta ocorrência."
Ocorrência RESOLVIDA não recebe apoio	`ApoioService`	"Esta ocorrência já foi resolvida."
Nome e e-mail obrigatórios; e-mail com formato válido	`UsuarioService`	"Informe nome e um e-mail válido."
E-mail único	`UsuarioService`	"Já existe um usuário com este e-mail."
Tipo é CIDADAO ou AGENTE	`UsuarioService`	"Tipo de usuário inválido."

Como o sistema não tem login, toda operação que depende de permissão recebe o id do usuário que está agindo. O service busca esse usuário no banco e consulta os métodos `podeApoiar()`, `podeMudarStatus()` e `podeEditar()`.

As regras ficam no service mesmo quando o front já esconde o botão, porque a rubrica avalia a regra no back.

Rotas da API

O pacote `api` sobe um servidor HTTP na porta 8080, com um handler por rota. O handler lê o pedido, chama o service e devolve JSON; ele não contém regra de negócio nem SQL.

Método e rota	Corpo ou parâmetros	Devolve
`GET /ocorrencias`	Opcionais: `?bairro=2&status=ABERTA`	Lista de ocorrências
`GET /ocorrencias?id=5`		Uma ocorrência
`POST /ocorrencias`	`titulo`, `descricao`, `endereco`, `idCategoria`, `idBairro`, `idUsuario`	A ocorrência criada
`PUT /ocorrencias?id=5&usuario=2`	`titulo`, `descricao`, `endereco`, `idCategoria`, `idBairro`	A ocorrência atualizada
`PUT /ocorrencias/status?id=5&usuario=7`	`status`	A ocorrência atualizada
`DELETE /ocorrencias?id=5&usuario=2`		Mensagem de confirmação
`GET /usuarios` e `GET /usuarios?id=3`		Lista ou um usuário
`POST /usuarios`	`nome`, `email`, `tipo`, `idBairro`	O usuário criado
`PUT /usuarios?id=3`	`nome`, `email`, `tipo`, `idBairro`	O usuário atualizado
`DELETE /usuarios?id=3`		Mensagem de confirmação
`GET`, `POST`, `PUT`, `DELETE /bairros`	`nome`; `?id=` para alterar e excluir	Lista, item ou confirmação
`GET`, `POST`, `PUT`, `DELETE /categorias`	`nome`; `?id=` para alterar e excluir	Lista, item ou confirmação
`POST /apoios`	`idUsuario`, `idOcorrencia`	Mensagem de confirmação
`DELETE /apoios?usuario=1&ocorrencia=5`		Mensagem de confirmação
`GET /relatorios?tipo=bairro`	`tipo`: `bairro`, `categoria`, `ranking` ou `status`	Lista de linhas do relatório

O parâmetro `usuario` identifica quem está agindo, já que não há login. O front envia o id escolhido no seletor "Estou usando como:".

Formato do JSON

Uma ocorrência, como o front espera receber:

```
{
  "id": 1,
  "titulo": "Buraco grande na Rua das Acácias",
  "descricao": "Buraco com cerca de 1 metro",
  "endereco": "Rua das Acácias, 120",
  "status": "ABERTA",
  "dataAbertura": "2026-11-02",
  "autor": { "id": 2, "nome": "Carlos Eduardo Lima" },
  "categoria": { "id": 1, "nome": "Buraco na via" },
  "bairro": { "id": 2, "nome": "Jardim das Flores" },
  "totalApoios": 5
}
```

Um usuário:

```
{
  "id": 7,
  "nome": "Patrícia Mendes",
  "email": "patricia.mendes@exemplo.com",
  "tipo": "AGENTE",
  "bairro": { "id": 1, "nome": "Centro" }
}
```

Uma linha de relatório (`tipo=bairro` , `categoria` ou `status`) e uma linha do ranking:

```
{ "nome": "Centro", "total": 3 }
```

```
{ "id": 1, "titulo": "Buraco grande na Rua das Acácias", "bairro": "Jardim
das Flores", "status": "ABERTA", "totalApoios": 5 }
```

Detalhes do servidor

- **CORS:** toda resposta leva `Access-Control-Allow-Origin: *` , `Access-Control-Allow-Methods` e `Access-Control-Allow-Headers` , e o método `OPTIONS` responde 204. Sem isso o navegador bloqueia o `index.html` aberto direto do disco.
- **Codificação:** ler e escrever em UTF-8, com `Content-Type: application/json; charset=utf-8` , para os acentos não quebrarem.
- **JSON sem biblioteca:** se o Gson não foi visto em aula, a saída é montada pelos métodos `toJson()` e a entrada é lida por uma classe utilitária simples, já que os corpos enviados pelo front são objetos de um nível só.

- **Datas:** sempre no formato `AAAA-MM-DD`.
- **Qual servidor:** este documento assume o `HttpServer` do próprio JDK. Se vocês viram Servlets ou Spring até a fase 7, use o que foi ensinado e mantenha as mesmas rotas e o mesmo JSON.

Tratamento de erros

Nenhum erro pode derrubar o servidor nem chegar ao usuário como texto técnico. O handler captura as exceções e sempre responde com `{"erro": "mensagem"}`.

Situação	Exceção	Status HTTP	Mensagem
Regra de negócio violada ou campo inválido	`RegraNegocioException`	400	A da tabela de regras
Id que não existe	`NaoEncontradoException`	404	"Ocorrência não encontrada." (ou usuário, bairro, categoria)
JSON malformado ou número inválido no pedido	`RegraNegocioException`	400	"Dados enviados em formato inválido."
Valor duplicado em coluna `UNIQUE`	`DadosException`	400	"Já existe um registro com este valor."
Exclusão de registro em uso por outra tabela	`DadosException`	400	"Não é possível excluir: existem ocorrências ligadas a este registro."
Banco fora do ar ou credenciais erradas	`DadosException`	500	"Não foi possível acessar o banco de dados. Tente novamente."
Método não suportado na rota	Nenhuma	405	"Método não suportado."
Qualquer erro inesperado	`Exception`	500	"Erro interno. Tente novamente."

- **Três exceções próprias:** `RegraNegocioException`, `NaoEncontradoException` e `DadosException`, em um pacote `exception` ou dentro de `service`.

- **`DadosException` guarda a causa:** o DAO embrulha a `SQLException` original, e o detalhe técnico vai para o console do servidor, não para a tela.
- **Erros do Oracle:** o código da `SQLException` diferencia os casos: 1 para valor duplicado e 2292 para registro em uso.
- **Sucesso:** 200 para consultas e alterações, 201 para criação.

Critérios de aceite

Antes de liberar o back para a integração com o front:

- [ ] O projeto compila e o `Main` sobe o servidor na porta 8080 seguindo só o README.
- [ ] A conexão usa o `db.properties`, e nenhuma senha está no código ou no repositório.
- [ ] Inserir, listar, buscar, atualizar e excluir funcionam para ocorrências e usuários, gravando no Oracle.
- [ ] `GET /ocorrencias` devolve as 12 ocorrências da carga, com autor, categoria, bairro e total de apoios.
- [ ] Os filtros por bairro e por status funcionam juntos e separados.
- [ ] Um cidadão tentando mudar o status recebe erro 400 com a mensagem certa.
- [ ] Apoiar duas vezes a mesma ocorrência e apoiar a própria ocorrência são recusados.
- [ ] Cadastrar um usuário com e-mail repetido devolve mensagem clara.
- [ ] As quatro rotas de relatório devolvem os resultados esperados da especificação do banco.
- [ ] Com o banco desligado ou a senha errada, o servidor responde com mensagem e continua no ar.
- [ ] Os acentos aparecem corretos no JSON.
- [ ] As classes estão nos pacotes `model`, `dao`, `service` e `api`, sem SQL fora do `dao`.
- [ ] A dupla do pitch foi avisada de que as classes estão estáveis para o diagrama UML.